



HYTREX

CLARITY. PURPOSE. IMPACT.



DevUP

Propuesta de desarrollo

El gestor del desarrollo del proyecto

INGENIERÍA A CARGO

Juan Felipe Medina Orjuela

Juan Esteban Bonilla

Carlos Fernando Cáceres

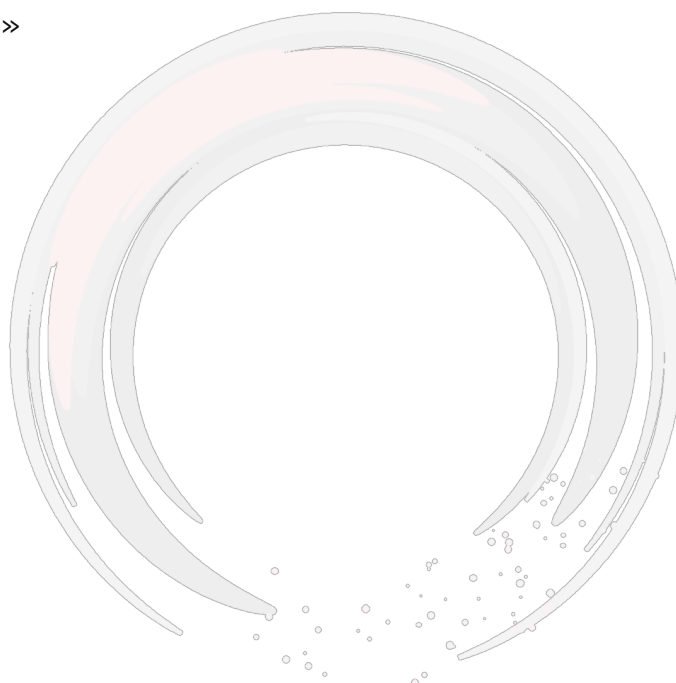
Resumen

DevUP es una plataforma para equipos que construyen software. No gestiona la representación del trabajo —tarjetas que alguien mueve a mano— sino el sitio donde el trabajo ocurre: el repositorio, la base de datos, el despliegue, la conversación y, cada vez más, el agente.

Este documento tiene dos mitades. La primera cuenta qué es DevUP, de dónde salió la idea, qué problema ataca —con las cifras que lo respaldan y sus fuentes—, a quién va dirigido y cómo pensamos sostenerlo. La segunda es el plan de desarrollo: todo lo que hay que hacer para que la plataforma exista de verdad, en web, en escritorio y en móvil.

No lleva fechas a propósito. Lleva actividades, en el orden en que conviene hacerlas y con el motivo de ese orden, que es lo que hace falta para repartir el trabajo entre tres personas.

«El enemigo de este producto no es una empresa. Es la pérdida de contexto entre ventanas.»



DevUP

1. De dónde sale esto

La idea no salió de un estudio de mercado. Salió de nuestra propia semana.

Trabajamos en remoto y trabajamos bien: el trabajo remoto funciona, y desde que la inteligencia artificial entró en el día a día, muchas cosas que antes costaban una tarde cuestan diez minutos. Pero al mirar de cerca en qué se va el tiempo aparecía siempre lo mismo, y no era escribir código.

Era el ir y venir. El repositorio en una pestaña, la base de datos en otra, el gestor de tareas en una tercera, la conversación en una cuarta, el panel del proveedor de despliegue en una quinta, y el asistente en una sexta al que hay que volver a explicarle el proyecto cada vez. Cada salto es barato; la suma no. Y lo que se pierde en cada salto no es solo tiempo: es el contexto, que hay que reconstruir del otro lado.

Nos pareció que eso era un problema con forma de producto. No un gestor de proyectos más —de esos hay muchos y son buenos— sino algo que reuniera en un solo sitio las ventanas entre las que se pierde el hilo.

2. El problema, y lo que se puede medir

La sensación es nuestra, pero la magnitud está estudiada. Tres investigaciones independientes describen las tres caras del mismo problema.

El coste de saltar entre aplicaciones

Un estudio publicado en Harvard Business Review en 2022 instrumentó a 137 trabajadores de veinte equipos en tres empresas del Fortune 500 durante cinco semanas. El resultado: una persona cambia de aplicación o de pestaña unas **1.200 veces al día**. Cada cambio cuesta poco más de dos segundos, pero el acumulado son **casi cuatro horas a la semana** —el **9 % del tiempo laboral**— dedicadas solo a reorientarse después de cambiar de sitio. [1]

El coste de recuperar el hilo

Mucho antes, en 2005, Gloria Mark, Víctor González y Justin Harris observaron en detalle a 24 trabajadores del conocimiento y describieron el trabajo real como **fragmentado por norma**: la gente pasa poco tiempo seguido en una misma esfera de trabajo antes de cambiar, y el **57 % de esas esferas se interrumpen**. Reanudar no es gratis: hay que volver a montar en la cabeza dónde se estaba. [2]

Lo que cambió la inteligencia artificial, y lo que no

La encuesta a desarrolladores de Stack Overflow de 2025 muestra las dos cosas a la vez. La adopción es casi total: el **84 %** usa o piensa usar herramientas de IA, frente al 76 % del año anterior. Pero la confianza cae: solo el **29 %** confía en que la respuesta sea correcta, frente al 40 % de 2024, y hay más gente que desconfía activamente (**46 %**) que gente que confía. El dolor concreto que más se repite lo dice el **66 %**: soluciones que están casi bien y que cuesta más depurar que escribir. Y cuando se pregunta por qué acudirían a una persona, la primera razón, con el **75 %**, es «cuando no me fío de lo

que responde la IA». [3]

Lo que junta las tres

Leídas juntas cuentan una historia sola. La IA no eliminó el cuello de botella: lo movió. Ya no está en escribir el código; está en **sostener el contexto** —para que el agente y la persona sepan de qué proyecto se habla— y en **verificar** lo que sale. Y las dos cosas ocurren hoy repartidas entre ventanas que no se hablan entre sí.

Ahí es donde queremos estar.

3. Qué es DevUP

«No somos un gestor de proyectos. Somos el gestor del desarrollo del proyecto.»

La diferencia no es un juego de palabras. Un gestor de proyectos administra la representación del trabajo: tarjetas que describen algo que está pasando en otro sitio, y que alguien tiene que acordarse de mover. La tarjeta y el código no se tocan nunca.

DevUP vive en el otro lado: el repositorio, la base de datos, el despliegue, el agente y la conversación, en el mismo sitio, de manera que el estado se deduzca de lo que pasó y no de lo que alguien anotó. Una integración continua en rojo abre una tarea sola. Un despliegue se ve sin entrar al panel del proveedor. Un agente trabaja con las credenciales de la organización y su resultado lo revisa el equipo.

De ahí sale la estrategia comercial, y conviene decirla explícitamente: **somos aliados, no competencia**. GitHub, Supabase, Vercel y los gestores de proyectos son el sustrato sobre el que corre DevUP. Si el enemigo es la dispersión, cada plataforma integrada es una victoria, no un rival.

4. A quién va dirigido

A cualquier organización que construya software. Eso incluye dos perfiles que se parecen menos de lo que parece: la **empresa de desarrollo**, que vive de entregar proyectos a terceros y necesita separación estricta entre clientes; y la **empresa con un área de desarrollo**, que construye para sí misma y necesita que ese área se entienda desde fuera.

El aislamiento entre organizaciones no es una función más para el primer perfil: es la condición para poder usarlo. Por eso está construido en la base de datos desde el primer día y no como un filtro en el código.

Y hay un tercer perfil que no paga y que importa igual: la persona que trabaja sola o con dos amigos. Es quien más sufre la dispersión y quien menos herramientas tiene para pelearla.

5. Qué existe hoy

DevUP no empieza de cero. Escrito a partir del código y no de lo que prometían los planes: hay **134 puntos de API** en 18 módulos, **45 tablas** —44 con política de aislamiento; la que falta es el registro de migraciones, que no guarda datos de nadie—, **24 migraciones**, **21 pantallas** y **231 comprobaciones automáticas** en verde en cada cambio: 172 de aislamiento entre organizaciones, 13 del socket del mundo, 26 del criterio de migraciones y 20 del diagnóstico de integraciones.

De las tres promesas del producto, dos están completas y en producción: el **espacio de trabajo** —organizaciones, canales, mensajería, llamadas con voz, vídeo y pantalla compartida, grabación con consentimiento, biblioteca de archivos, tablero de tareas, búsqueda global— y el **control de ventas** —servicios, clientes, embudo, cotizaciones y objetivos—.

La tercera —infraestructura— también está en pie, y es la novedad. Sobre la bóveda de credenciales cifrada —con rotación de su clave maestra— hay ya tres piezas desplegadas: la **vista de entornos y despliegues**, que pregunta al proveedor en vez de desplegar; la **base de datos como código**, que lee las migraciones del repositorio del cliente y las pasa por el criterio; y las **integraciones guiadas**, que dicen qué se está resolviendo a mano y qué lo ahorraría, con el archivo y la línea delante.

Lo que no existe todavía de esa tercera promesa son los **agentes**, y la segunda mitad de las integraciones: hoy se diagnostica, y montar la integración necesita credenciales del proveedor. Existe además **DevVerse**, el espacio recorrible con avatares, con su cartelera de estados, el encuentro por cercanía y la pizarra compartida ya funcionando.

Lo que falta es lo que ocupa la segunda mitad de este documento.

6. Cómo se sostiene

La idea es sencilla: **quien trabaja solo o casi solo, no paga**. Un equipo de hasta cuatro personas tiene la plataforma completa sin coste. No es generosidad: es el canal de entrada. Ese equipo es exactamente quien más siente el problema, y es quien lo va a contar cuando crezca.

A partir de ahí, por persona y mes. Los precios de abajo son una **estimación para validar**, no una tarifa cerrada.

Plan	Precio	Qué incluye
Libre	0 USD	Hasta 4 personas, una organización. Espacio de trabajo completo, llamadas, archivos, tareas y DevVerse. Un conector.
Equipo	12 USD / persona / mes	Sin tope de personas. Conectores sin limite, bóveda de credenciales, integraciones guiadas y vista de infraestructura.
Empresa	24 USD / persona / mes	Agentes, roles y auditoría, base de datos como código, inicio de sesión corporativo y soporte.
Consumo de agente	Medido, aparte	Se factura lo que consume, con el gasto visible por organización.

Por qué esos números

Las herramientas que DevUP reúne se pagan hoy por separado, y cada una está en el orden de entre 7 y 20 dólares por persona y mes: el gestor de tareas, la mensajería, el espacio virtual, el asistente de código. Doce dólares por el conjunto queda por debajo de la suma de lo que sustituye, que es la única comparación que un comprador hace de verdad. El salto a Empresa lo justifican tres cosas que solo importan cuando hay varios clientes o varios equipos: los agentes, la auditoría y el aislamiento verificable.

El consumo de agente va aparte por honestidad y por supervivencia: tiene un coste variable real por cada petición. Meterlo en una tarifa plana obliga a una de dos, y las dos son malas: encarecer a todos para cubrir a los pocos que lo usan mucho, o poner un tope silencioso que la gente descubre el peor día.

Queda para más adelante, y como idea sin cerrar, la venta de artículos cosméticos para DevVerse. Hoy no entra en el plan.



Plan de desarrollo

7. El punto de partida: la interfaz creció por acumulación

Antes de decidir qué añadir conviene decir con precisión qué pasa con lo que ya hay, porque cambia el orden del trabajo.

El **sistema visual está bien hecho**: hay cuatro niveles de superficie, tres familias tipográficas con un trabajo cada una, curvas y duraciones de animación con nombre, tres materiales y tres reglas escritas que explican el resto. Está atendido el movimiento reducido y la transparencia reducida. Eso no se toca.

Lo que no existe es **la capa de encima**: el armazón, el marco de página y la navegación entre pantallas. Cada funcionalidad llegó como una pantalla completa e independiente, y ninguna llegó como una pieza dentro de un marco, porque el marco nunca se escribió. Dieciséis veces seguidas.

Buena parte de esto ya está hecho, y conviene decir cuál. Existen el armazón de organización, el marco de página, el diálogo de confirmación, el desplegable, el área de texto y una capa de datos con caché. Las **ocho acciones irreversibles** ya no se deciden en el cuadro gris del sistema operativo; los **doce desplegables y áreas de texto** escritos a mano son ahora primitivas; y las **cinco cabeceras copiadas** son una sola.

Lo que sigue pendiente del diagnóstico son dos cosas, y son las dos más caras: las **88 llamadas a la API sueltas** en pantallas, que hay que ir migrando a la capa nueva una por una, y las **pantallas que crecieron sin dividirse** —la de ventas mide 1.273 líneas—, que no se pueden partir con seguridad mientras no haya pruebas que abran un navegador.

El responsive sigue siendo el punto débil, aunque menos que al escribir esto: la navegación ya es un cajón en pantalla estrecha en vez de una barra fija que se come dos tercios del ancho. Lo que falta es el resto — las pantallas de dentro se diseñaron para un monitor, y que la barra se aparte no las arregla.

«No el estilo —que está bien— sino el armazón que nunca se construyó debajo. Escrito antes de construirlo; hoy está en pie y lo que queda es mudarse a él.»

8. Lo que hay que hacer

En orden, y con el motivo del orden. La regla es siempre la misma: primero lo que protege lo que ya existe, después lo que abarata lo siguiente, y al final lo que solo hace falta cuando duela.

A · Que nada se pierda

Nada de esto añade una función y todo esto evita perder lo que ya hay: usuarios reales, una organización con su historia dentro, grabaciones de llamadas y credenciales de terceros cifradas.

Las copias ya existen, con su restauración probada y su procedimiento de rotación de la clave maestra. Lo que queda de este bloque es lo que no puede hacer un script: llevarse el archivo con los secretos fuera de la máquina, y dar de alta un servidor de correo. Y una lección que costó dos días de respaldos

perdidos: tener el script y tener la copia no son lo mismo, y solo la segunda cuenta — el planificador fallaba en silencio en cada reinicio y nadie se enteraba.

- Copias de seguridad de la base de datos y del almacén, fuera de esa máquina, con retención y con una restauración probada. Una copia que nadie ha restaurado no es una copia.
- Registrar el ejecutor de despliegue, que está descargado y sin configurar, de modo que los despliegues dejen de hacerse a mano.
- Servidor de correo real. Hoy las invitaciones y las recuperaciones de contraseña se escriben en el registro del servidor en vez de enviarse: sirve para probar y no sirve para usar.
- Custodia y procedimiento de rotación de la clave maestra de la bóveda. Si se pierde, todas las credenciales guardadas quedan indescifrables para siempre.

B · Cerrar lo que quedó a medias

Cosas empezadas que hoy cuestan más abiertas que cerradas: fusionar el arreglo de servidor que lleva días en una rama, levantar o contratar el servidor de retransmisión que hace que las llamadas se oigan fuera de una red local, sacar la integración de música del modo de desarrollo, y explicar en la interfaz por qué falla a veces añadir un repositorio —que casi nunca es un fallo nuestro, pero lo parece—.

C · Pruebas que abren un navegador

La integración continua ya cubre lo difícil en cada cambio: tipos, migraciones, aislamiento entre organizaciones y construcción. Lo que no cubre es nada que abra un navegador, y ahí es donde viven los fallos que ninguna prueba de tipos puede ver. El ejemplo lo tenemos: el almacén de archivos nunca llegaba a crearse y pasó meses sin detectarse, porque solo se manifestaba a mitad de una subida real.

La trampa a evitar: una prueba de navegador que falla a veces es peor que ninguna, porque enseña al equipo a ignorar el rojo. Mejor cinco estables que veinte inestables.

I · La interfaz: almacén, marco, datos y móvil

Es la respuesta al diagnóstico del apartado 7, y va antes de las pantallas nuevas por un motivo de coste: la pantalla nueva más grande que queda, si se construye hoy, sale con su séptima cabecera copiada, su propio código de carga, su propio desplegable y su propio aviso de error. Y después habrá que migrarla igual, con la diferencia de que entonces estará en producción.

- **Almacén de organización**, con la misma factura que el del espacio de trabajo, y naciendo con cajón para móvil en vez de con una barra fija que luego haya que desmontar.
- **Marco de página**: un componente que recibe título, rótulo, icono y acciones, y mata las cinco cabeceras copiadas. Con él, los tres finales posibles de una carga —cargando, fallo, vacío— dejan de improvisarse en cada pantalla.
- **Las primitivas que faltan**: desplegable, área de texto, menú, pestañas, tabla y, sobre todo, el diálogo de confirmación que sustituye a los ocho cuadros del sistema operativo. Es el punto con mejor relación entre esfuerzo y resultado de todo el plan.
- **Capa de datos**: hoy hay 88 llamadas a la API y 71 efectos escritos directamente en las pantallas, sin caché, así que volver a una pantalla la recarga entera. Con una pieza pequeña y propia se acaba esa repetición y se escribe de paso la regla de errores: el error de un campo va junto al campo, el de una acción va en un aviso flotante.

- **Partir las pantallas grandes.** La de ventas tiene 1.273 líneas y contiene la pantalla, sus subcomponentes, sus formularios y su estado. No se puede reutilizar, ni probar por separado, ni tocar entre dos personas la misma semana.
- **Muestrario y teclado:** una pantalla de desarrollo con todas las primitivas en todos sus estados —es donde se ve que dos componentes han derivado antes de que lo vea un usuario— y una paleta de comandos que hace que navegar dieciséis pantallas deje de doler.

D · Infraestructura — desplegada

Entornos y despliegues en una sola pantalla, sobre la bóveda que ya existe. Ya se puede añadir un entorno, apuntarlo a un repositorio y ver en qué estado quedó lo último que entró, con su commit, su autor y un enlace al registro.

Pregunta a GitHub y no a un proveedor de alojamiento, que es coherente con la decisión de orquestar en vez de desplegar: para la mayoría de equipos la plataforma que ya usan es Actions, y su credencial ya estaba en la bóveda. Lo que falta es el segundo proveedor —que es donde se comprobará si la traducción de estados aguanta— y encender con esto los muebles de DevVerse que hoy son decorado: la pantalla de despliegue y el rack de servidores.

Integraciones guiadas — el diagnóstico, desplegado

La pieza más diferenciadora del producto. La forma corta es esta: «Estás guardando sesiones a mano. Supabase te da autenticación, base de datos y almacenamiento. ¿Lo monto?» — y si la persona dice que sí, el trabajo ocurre detrás: crear el proyecto, guardar las claves en la bóveda, escribir el esquema, conectar el protocolo de contexto y avisar.

La primera mitad ya está. DevUP lee el repositorio conectado y dice qué se está resolviendo a mano —autenticación propia, archivos en el disco del servidor, base de datos sin migraciones, un archivo de credenciales dentro del repositorio—, y cada recomendación enseña el archivo y la línea donde se ve. Sin esa prueba delante sería publicidad dentro de una herramienta de trabajo. Lo que falta es el «¿lo monto?», que necesita credenciales del proveedor.

El diagnóstico de partida está infravalorado: mucha gente no ha descartado esas herramientas, es que **no sabe que existen**. Un catálogo donde buscas lo que ya sabes que quieres no arregla eso; un producto que dice «para lo que estás haciendo, esto te ahorraría tanto», sí.

Agentes

Aquí sí hay carrera y va rápida. Pero las herramientas que existen son de escritorio y para una persona: no tienen organización, ni roles, ni canales, ni bóveda compartida, ni aislamiento entre clientes. DevUP tiene las seis cosas hechas y probadas.

El hueco es **el agente en equipo**: no «un agente que trabaja por mí» sino uno que trabaja para la organización, con sus credenciales, y cuyo resultado ve y aprueba el equipo. Antes de escribir código hay que decidir dos cosas: qué puede tocar un agente y con qué credenciales. La respuesta al primero probablemente sea el aislamiento por copia de trabajo, que es lo que hace que dos agentes no se pisen; la del segundo es la bóveda, y por eso conviene que llegue después de que la vista de infraestructura la haya puesto a prueba con algo más que dos conectores.

Y una regla de producto que no es negociable: **el agente propone y la persona aprueba**. El cambio se revisa dentro de DevUP antes de que salga.

Un recopilador de contexto

Notas enlazadas entre sí, con grafo, donde vive el porqué de las cosas. Lo que no puede ser es otra aplicación de notas que nadie rellena: ese es el final de casi todas, porque escribir la nota es trabajo extra y su beneficio llega meses después y a otra persona.

Lo que aquí lo hace distinto es que el contexto **ya existe y está disperso**: decisiones discutidas en canales, tareas con su conversación, repositorios con sus commits, entornos con sus despliegues. Lo que no hay es un hilo que los una. Así que la nota no se escribe, **se deriva** — es un nodo que enlaza cosas que ya pasaron, y el trabajo de la persona es confirmarla y titularla, no redactarla.

El agente que cada equipo prefiera, por MCP

Que DevUP hable por MCP con el motor agéntico que el equipo ya usa —Claude, ChatGPT, el que sea— y que ese agente genere entrevistas, proponga un roadmap, prepare reuniones y asigne tareas.

Esto invierte una decisión anterior, y a mejor. Lo previsto era guardar una clave de API del modelo en la bóveda y llamarlo nosotros, lo que nos convierte en intermediarios de un servicio que no controlamos: su factura, su elección de modelo y su responsabilidad cuando se equivoca. Al revés, DevUP es un **servidor** MCP y no un cliente de un modelo: expone herramientas —leer un canal, listar tareas, abrir un pull request, mirar un entorno— y el agente lo pone el equipo con su propia suscripción.

La regla que ya estaba decidida encaja sola: **el agente propone y la persona aprueba**, y esa aprobación ocurre dentro de DevUP, que es donde está el equipo. El agente vive fuera; la superficie donde se decide, dentro. Y el renglón de consumo de agente de la tabla de precios desaparece.

El trabajo de verdad no es el protocolo: es que cada herramienta pase por el mismo aislamiento que la API, con la identidad de una persona y bajo sus políticas. Un servidor que consulte con el rol de la aplicación se salta el aislamiento entero y le enseña a un agente los datos de todas las organizaciones.

Un diseñador propio de bocetos y MVP, que produce código

Un lienzo dentro de DevUP para hacer bocetos y MVP —como se haría en cualquier herramienta de diseño— y, cuando el diseño está listo, que la IA lo pase a código.

La trampa, dicha antes de empezar: pedirle a un modelo que convierta píxeles en código sale siempre igual de mal, y no por falta de talento del modelo — **la información no está ahí**. Un rectángulo gris con texto dentro puede ser un botón, una etiqueta o una tarjeta, y ninguna cantidad de inteligencia lo saca de la imagen con certeza. Es el mismo fallo que ya conocemos de otro sitio: lo que no se guarda no se puede adivinar después.

Por eso el lienzo tiene **dos capas**. Una libre —rectángulos, texto, marcos— para explorar, que es para lo que sirve un boceto. Y una semántica: cualquier cosa del lienzo se puede **ascender a componente** del sistema, que ya existe entero y está en un muestrario. Ahí es donde la IA hace lo que de verdad sabe hacer: no generar maquetación a partir de una imagen, sino **proponer el ascenso** —«esto parece un botón primario, esto una lista de tarjetas»—. Es una correspondencia difusa contra un catálogo cerrado, que es donde un modelo acierta mucho y equivocarse cuesta un clic.

Con el diseño ascendido, generar el código deja de ser una traducción: es imprimir un árbol de componentes que ya existen. No hay pérdida porque no hay conversión. Y un diseño hecho así **no puede salirse del sistema visual** — que es justo el problema que costó dieciséis pantallas—, además de enterarse el día que el sistema cambie, cosa que un archivo de diseño no hace nunca.

Reorganizar la superficie por sectores

Hay veintiuna pantallas y todas valen lo mismo: una lista plana en una barra. Pero un desarrollador, un diseñador, un scrum master y un product owner no usan las mismas cinco cosas ni en la misma proporción, y hoy los cuatro buscan lo suyo entre lo de los demás.

La pieza es **partir la pantalla**: elegir trabajar en una, dos o tres zonas y poner en cada una la herramienta que toque —el editor y el canal, el tablero y la pizarra, el diseño y su código al lado—. Y el rol es **un preajuste de disposición, no un muro**: decide qué sale primero, no qué se puede abrir. Si impide entrar a algo, lo que se ha construido es un sistema de permisos que nadie pidió.

Tiene precedente en casa, que es la mejor señal de que es alcanzable: el panel personal ya hace esto en pequeño —celdas y no píxeles, arrastrar y estirar, guardado por persona, colapso a una columna en pantalla estrecha—. O la partición usa esa misma primitiva, o acabaremos con dos sistemas de disposición que se parecen y no son iguales.

Base de datos como código — desplegada

Las migraciones del cliente, leídas de su repositorio y pasadas por el mismo criterio que aplicamos aquí: solo se añaden, se pueden aplicar dos veces, y la política de aislamiento es parte de la migración y no un paso aparte. Ese criterio es el producto, no un detalle interno: se aprendió a base de un fallo silencioso que costó una migración entera encontrar.

Se lee el texto y no se ejecuta nada — ni se conecta a la base del cliente ni se corre una sola sentencia. Y el analizador se probó contra nuestras propias veinticuatro migraciones, que es el único banco de pruebas honesto que había: si a las que sí cumplen el criterio les pone un error, el equivocado es el analizador. Pasó dos veces, y las dos se corrigió el criterio.

Identidad propia y apertura

Identidad de DevUP y la apertura a gente de fuera del equipo. No se puede empezar sin el bloque A: abrir a usuarios ajenos un sistema sin copias de seguridad y sin correo real no es una beta, es pedirle a alguien que se registre por un enlace que no le llega para guardar su trabajo en un disco que nadie respalda.

Escalar, cuando toque

Nada de esto hace falta hoy y hacerlo antes de tiempo es trabajo tirado. Va escrito para que cuando duela se sepa dónde mirar: un almacén compartido para presencia y límites de peticiones, el día que haya una segunda instancia de la API; y el reparto de la malla de voz dentro de una misma sala, que hoy resuelve veinte personas en cuatro salas pero no doce en una.

9. Las tres superficies

La plataforma tiene que existir en web, en escritorio y en móvil. No son tres proyectos.

Web, con el responsive bien hecho

Es la base y es requisito de las otras dos. Hoy no existe, y por eso el armazón del bloque I nace con cajón en vez de con una barra de ancho fijo: construirlo dos veces es justo lo que este plan existe para evitar.

Escritorio

Envolver la web para tener un icono no vale la pena. Lo que solo puede hacer una aplicación de escritorio, sí: **abrir los repositorios de verdad en el disco y ejecutar procesos de verdad**, en vez del entorno embebido que corre dentro del navegador con sus límites. Ese es el salto, y es además donde está la competencia. Añade también notificaciones del sistema —que la integración continua en rojo llegue sin tener la pestaña abierta— y presencia real.

Móvil

La misma tecnología compila a móvil, así que la web bien hecha es la mayor parte de las tres. Conviene saber que el soporte móvil de esa tecnología es bastante más joven que el de escritorio, y probarlo con algo pequeño antes de apostar el calendario.

Y conviene tener claro cuál es la función móvil que de verdad importa, porque no es «todo más pequeño»: es **aprobar**. El agente propone, la persona va en el autobús y da el visto bueno. Es corta, es frecuente y es exactamente lo que un teléfono hace bien.

Lo que no baja a móvil es DevVerse. Un espacio recorrible en un teléfono no es la misma cosa hecha más pequeña.

10. Cuatro decisiones técnicas ya tomadas

Cuatro puntos del proyecto se estudiaron a fondo porque condicionan lo que se puede construir. Quedan cerrados así.

La música compartida no puede sonar sincronizada entre plataformas

No es cuestión de esfuerzo. Reproducir en el navegador exige una suscripción de pago por oyente en un servicio, un programa de desarrollador de pago en otro, y en un tercero no hay interfaz oficial. No existe ninguna forma de que dos personas en servicios distintos estén en el mismo segundo de la misma canción.

Lo que sí se puede, y es casi todo lo que se quería: la cola es **agnóstica**. Se guarda la canción —su identificador internacional de grabación, título y artista—, no el enlace de un servicio, y cada persona la reproduce en el suyo. Se gana una sola lista de verdad; se pierde la escucha simultánea al segundo. Y guardar la canción en vez del enlace es lo correcto aunque solo hubiera un servicio: el enlace es de una plataforma, la canción es del equipo.

El encuentro por cercanía no rompe el cifrado

Las llamadas van cifradas de extremo a extremo y esa decisión no se reabre. La idea de que acercarse a alguien encienda la cámara automáticamente sí chocaba con ella: en un espacio con mucha gente el vídeo obliga a un servidor de medios en medio, que es exactamente lo que rompería el cifrado.

La versión acordada no tiene ese problema, y además es mejor diseño. Acercarse abre un menú —saludar o llamar—, y las cámaras se encienden solo si los dos aceptan. Una llamada individual son dos navegadores y una sola conexión, cifrada por definición, sin nada en medio. La pizarra compartida va por el canal de datos de esa misma conexión, así que **el servidor nunca ve lo que se dibuja**, y guardarla funciona como ya funciona grabar: uno de los dos exporta y sube el resultado.

La gamificación se puntúa por ritos de calidad, no por volumen

Puntuar volumen —confirmaciones de código, líneas, tareas cerradas— produce exactamente lo que se puntúa: muchas confirmaciones pequeñas, tareas troceadas, y la persona que arregló el fallo difícil en tres líneas la última de la tabla.

Se puntúan en cambio los ritos que cuestan y que nadie quiere hacer: una migración con su política de aislamiento y su caso de prueba, una revisión de código con comentarios de fondo, un fallo con prueba de regresión, la integración continua devuelta a verde. Sin clasificación pública individual, y con recompensa siempre cosmética: en cuanto los puntos abren puertas, dejan de ser un juego.

Eso resuelve además el riesgo obvio, que es que dos personas se monten una organización, se inventen tareas y se las marquen como hechas. La defensa no está en separar niveles por organización —eso reparte el problema en vez de impedirlo— sino en **qué acuña moneda**: solo un hecho que la plataforma pueda verificar contra un sistema externo que la persona no controla. Marcar una tarea no acuña nada. Una integración continua en verde sobre un repositorio real no se puede falsificar.

No construimos infraestructura de despliegue propia

Sería competir con los proveedores grandes en su terreno y con su curva de costes, y contradice el posicionamiento del apartado 3. Lo coherente es **orquestrar**: guardar la credencial en la bóveda, disparar el despliegue en la plataforma que el cliente ya usa, y enseñar el estado en una sola pantalla.

11. DevVerse

DevVerse es el espacio recorrible con avatares: salas, zonas, muebles, audio por cercanía. Conviene decir qué papel juega, porque es la parte del producto que más fácil se malinterpreta.

La categoría de oficinas virtuales existe desde hace años y no le ha ido bien a nadie: su referente se escindió en 2026 como pyme sin capital riesgo. Pero todas fracasaron prometiendo lo mismo —ser el sustituto de la oficina, «vente aquí a estar»—. DevVerse no promete eso. Es **la recompensa y la cara social de un producto de trabajo que ya funciona sin él**.

De ahí sale la decisión que ordena lo demás: la plataforma y DevVerse se separan en dos mundos, con un contrato mínimo entre ellos —**el trabajo genera puntos, los puntos se gastan en DevVerse**— y nada más cruza. Que el contrato sea pequeño es lo que hace la separación real y no un dibujo: hoy los dos comparten sesión, socket y media barra lateral, y por eso tocar uno rompe el otro. Es también lo que permite que la plataforma baje a móvil sin arrastrarlo.

Un hallazgo que cambia el presupuesto de todo este apartado: el mundo **ya está amueblado**. Hay 55 muebles dibujados, y entre ellos están la máquina recreativa, el fútbolín, la mesa de mezclas, el estante de discos, la pizarra, la vitrina de trofeos y el mobiliario completo de un apartamento. Y el avatar ya es un catálogo de índices por capas. Casi nada de lo que se quiere hacer pide dibujar un mundo nuevo: pide **enchufar el que hay**. La tienda de ropa, por ejemplo, es una tabla de desbloques sobre un catálogo que ya existe.

Lo que sí falta, y es el trabajo real: el menú al acercarse, la pizarra, la economía de monedas con su libro de cuentas, los atuendos guardados por organización, y la cartelera sobre cada personaje con nombre, rol y estado. De los estados, el que importa es el tercero —**ocupado, pero abierto a llamadas**—, que ninguna herramienta de trabajo tiene y que es la verdad la mayor parte del tiempo.

12. Riesgos, y lo que no se toca

- **Una tabla nueva sin política de aislamiento.** El aislamiento falla en silencio: no da error, devuelve cero filas y sigue. Ya pasó una vez y costó una migración encontrarlo. Toda tabla nueva necesita su política y su caso de prueba, y toda actualización que importe comprueba cuántas filas tocó, no que no haya excepción.
- **La clave maestra de la bóveda.** Si cambia o se pierde, todas las credenciales guardadas quedan indecifrables. Es el bloque A y va antes de que la bóveda guarde nada más.
- **El despliegue automático y la carpeta de trabajo.** Con el ejecutor activo, una edición a medio hacer puede irse a producción sin estar confirmada. O se asume, o el despliegue pasa a clonar aparte.
- **Pruebas de navegador inestables.** Cinco estables antes que veinte que fallan a veces.
- **La entrada al entorno de desarrollo embebido** necesita una navegación dura por el aislamiento que exige el navegador. Convertirla en un enlace normal lo rompe, y no es evidente por qué.

Y una zona restringida: el sistema visual base y el interior de DevVerse no se tocan de camino a otra cosa. Se tocan cuando se pidan, con las referencias delante.

13. Lo que falta decidir

De las cinco que había, tres se han cerrado solas al mirarlas de cerca. Las copias no necesitaban una unidad de red: todo lo que hay que salvar son dos megas, así que se copian a mano y ya. El servidor de voz no era una decisión sino un alta, porque auto-alojarlo es imposible con el despliegue de hoy —no se publica ni un puerto—. Y el tema claro entró, junto al oscuro y con conmutador.

- Los tres ritos concretos que acuñan moneda al empezar, y el techo semanal por persona. Es media hora de conversación y define la economía entera.
- El tamaño definitivo del avatar y cuántos cuerpos base hay de salida.
- Si la nota de contexto pertenece a la organización o al espacio de trabajo. Cambia la política de aislamiento, y esa no se cambia después.
- Si se retira la clave del modelo de la bóveda. Con DevUP como servidor MCP, guardar una credencial de un modelo deja de tener sentido.
- Qué herramientas MCP pueden escribir y cuáles no. Es la misma pregunta de qué puede tocar un agente, y por este camino tiene una respuesta más fácil.
- Si la clave del modelo para el diseño a código la ponemos nosotros. Es una función con coste acotado, a diferencia de un agente que trabaja todo el día: ahí sí tiene sentido incluirla y medir el consumo.
- Si los roles son preajustes de disposición o permisos. Si son permisos, esto se convierte en otro proyecto.

14. El orden, de un vistazo

Con una sola regla detrás: primero lo que protege lo que existe, después lo que abarata lo siguiente.

	Bloque	Por qué va ahí
1	Que nada se pierda	Protege lo que ya existe y son las horas más baratas del plan.
2	Cerrar lo a medias	Deuda que cuesta más abierta que cerrada.
3	Pruebas de navegador	La red que hace segura toda la migración de interfaz que viene detrás.
4	Interfaz: almacén, marco, datos y móvil	Es la mitad de las pantallas que quedan, escrita una vez en vez de siete.
5	Vista de infraestructura	Cierra la tercera promesa y enciende los muebles muertos de DevVerse.
6	Integraciones guiadas	Lo que nadie más hace y lo que se entiende en treinta segundos.
7	Agentes	Necesita la bóveda probada por el bloque anterior.
8	Base de datos como código	Se apoya en un criterio que ya existe y está probado.
9	Identidad propia y apertura	Necesita copias de seguridad y correo real: el bloque 1 entero.
10	Escalar	Solo cuando duela. Antes es trabajo tirado.



Bocetos del avatar

Las imágenes que siguen son **bocetos**, no el personaje definitivo ni el acabado con el que saldría. Están para fijar la dirección y para poder discutir sobre algo concreto en vez de sobre una descripción.

La decisión que ilustran sí es firme: el personaje se **modela en tres dimensiones** —con volumen, esqueleto y luz reales— y lo que entra al juego es un **render** de ese modelo. El motor sigue siendo el mismo lienzo de dos dimensiones que ya existe, así que ni el renderizador ni la red se enteran.

Esa tubería es lo que hace viable el catálogo que se quiere: las cuatro direcciones salen de girar el modelo y volver a renderizar, el andar sale del esqueleto en vez de dibujarse fotograma a fotograma, y una prenda nueva se modela una vez en vez de dibujarse a mano en cuatro vistas. Con diez o quince skins y varias piezas por accesorio, esa diferencia es la que decide si el catálogo es posible.

El render de estas páginas está hecho con volúmenes simples, así que la cabeza se ve cúbica y el pelo es un casquete. Con una herramienta de modelado real el mismo personaje lleva la forma esculpida y la silueta trabajada: **cambia la calidad del modelo, no la tubería ni lo que el juego recibe.**



Boceto · el modelo renderizado, ampliado ocho veces para ver el resultado del sombreado. Cada superficie recibe la luz según hacia dónde mira.



Boceto · las cuatro direcciones y dos fotogramas de andar. Ninguna se dibujó: son el mismo modelo girado y vuelto a renderizar.

Fuentes

- [1] Murty, R. N.; Dadlani, S.; Das, R. B. How Much Time and Energy Do We Waste Toggling Between Applications? Harvard Business Review, 29 de agosto de 2022. Estudio sobre 137 trabajadores de 20 equipos en tres empresas del Fortune 500, durante cinco semanas.
hbr.org/2022/08/how-much-time-and-energy-do-we-waste-toggling-between-applications
- [2] Mark, G.; González, V. M.; Harris, J. No Task Left Behind? Examining the Nature of Fragmented Work. CHI 2005, Portland, Oregón. Universidad de California en Irvine. Observación detallada de 24 trabajadores del conocimiento.
ics.uci.edu/~gmark/CHI2005.pdf
- [3] Stack Overflow. 2025 Developer Survey y Developers remain willing but reluctant to use AI.
survey.stackoverflow.co/2025/ai

Los datos sobre el estado del código de DevUP —número de puntos de API, tablas, migraciones, pantallas, comprobaciones de aislamiento y las cifras de la auditoría de interfaz del apartado 7— están contados sobre el repositorio, no estimados.

Nota de marca. El isotipo de estas páginas es una reconstrucción hecha para poder maquetar el documento; antes de enviarlo fuera conviene sustituirlo por el archivo original.